



CortexAI Current User (Restore Session) Flow Documentation

This document explains the setup, configuration, variables, functions, and the detailed file-to-file state flow showing how the active user's session is restored on page reload across CortexAI.

1. Flow Overview (For Beginners)

When a user visits the website, we don't want them to log in again if they have already signed in recently.

To do this:

1. On load, the frontend immediately requests the logged-in user's details from the Gateway's `/me` endpoint.
 2. The browser automatically attaches the stored secure `session` cookie containing the session ID.
 3. The Gateway reads this cookie and looks it up in Redis.
 4. If found, the Gateway returns the user details. The frontend saves these details in Redux to log the user in automatically.
-

2. Variables & Functions Dictionary

Frontend Components & Features

- `getuser` (function in `App.jsx`):
 - **What it does:** Triggered inside a React `useEffect` hook on app load. Calls the `getcurrentuser` helper function and dispatches the user data to Redux.
- `getcurrentuser` (helper function in `getcurrentuser.js`):
 - **What it does:** Sends a `GET` request to the Gateway at `/me`.

- **How it works:** Uses the custom Axios instance (which has `withCredentials: true`) to carry browser cookies. If the server returns a profile, it is returned.
- **Error Catching:** Contains an `error.response` check to handle issues like expired sessions, printing `Backend Error Data` and `Backend Status Code` to console before returning `null`.
- `setUserdata` (Redux action in `userslice.js`):
 - **What it does:** Updates the global state `userData` with the fetched profile data.

Gateway Verification & Response

- `protect` (middleware function in `auth.middleware.js`):
 - **Variables:**
 - `sessionId` : Extracted from the request cookie object (`req.cookies.session`).
 - `session` : Holds the stringified user object returned from the Redis cache query `redis.get("session-" + sessionId)`.
 - **How it works:** Validates the cookie session ID against Redis. If cached session data is found, it parses it and assigns it to `req.user` context.
 - **Error Catching:** Wraps the parsing logic in a try-catch block; if session serialization is corrupted, it catches `parseErr` and returns a `500` error response.
 - `getCurrentuser` (controller function in `user.controller.js`):
 - **What it does:** Returns `req.user` profile data as a JSON payload to the frontend.
-

3. Global Integration (Where, When, & How the Variables/Functions are Used)

The `getCurrentuser` flow connects different parts of the workspace to maintain session continuity:

A. Client-Side Startup Execution (`getCurrentuser` & `getuser`)

- **Where:** Used in `App.jsx` inside the React initialization stage.

- **When:** Runs automatically once when the browser loads or refreshes the page.
- **How:** `getuser` calls `getcurrentuser()` from features and receives the session profile data. If valid, it dispatches the profile details directly to Redux.

B. Gateway Route Protection (`protect`)

- **Where:** Imported and run inside `gateway/index.js` .
- **When:** Registered as a middleware interceptor on routes like `/me` , `/chat` , and `/agent` .
- **How:** When the client attempts to reach `/me` (via the `getcurrentuser.js` API call), the `protect` middleware runs first. It extracts the session ID from request headers, queries the Redis database, and validates the session. If validation succeeds, it binds the user profile context to `req.user` and permits the request to pass.

C. Server Response Handling (`getCurrentuser`)

- **Where:** Imported and run inside `gateway/index.js` .
- **When:** Invoked only after the `protect` middleware has verified the active user and populated `req.user` .
- **How:** Placed on route `app.get("/me", protect, getCurrentuser)` . This handler reads `req.user` and returns the JSON payload back to the client browser.

D. Frontend State Hydration (`setUserdata`)

- **Where:** Imported and used in `App.jsx` , `home.jsx` , and `sidebar.jsx` .
 - **When:** Called inside `App.jsx` immediately after receiving the response from `getcurrentuser()` .
 - **How:** Dispatches `setUserdata(data)` which writes the profile into Redux store variable `userData` .
 - **Details:**
 - **In `home.jsx` :** Checks `userData` . If populated, it automatically closes the login panel.
 - **In `sidebar.jsx` :** Extracts fields (`userData.name` , `userData.email` , `userData.avatar`) to show profile data in the workspace layout.
-

4. File-to-File getCurrentuser Workflow

Below is the step-by-step path the code execution takes from file to file during session restoration:

```
[Trigger] App.jsx (Frontend)
|  —> Component mounts. runs useEffect() hook.
|  —> Calls getCurrentuser() features helper.
▼
[API Request] getCurrentuser.js
|  —> Sends HTTP GET request to Gateway "/me" (attaching session cookie).
▼
[Gateway Entry] gateway/index.js (Gateway)
|  —> Matches path "/me" and executes protect middleware first.
▼
[Security Guard] gateway/middlewares/auth.middleware.js
|  —> Extracts sessionId from request cookies.
|  —> Runs: redis.get("session-" + sessionId) using shared/redis/redis.js.
|  —> Parses JSON string, binds to req.user, and calls next().
▼
[Endpoint Controller] gateway/controllers/user.controller.js
|  —> Runs getCurrentuser() controller.
|  —> Sends back req.user profile with status 200.
▼
[State Handler] App.jsx
|  —> Receives status 200 response containing user profile data.
|  —> Dispatches: dispatch(setUserdata(data)) to redux/userslice.js.
▼
[UI Updates] home.jsx & sidebar.jsx
    —> Detects userData in Redux is populated.
    —> Closes login popup modal and displays active workspace and user card.
```